

IMPERIUNS BOT

Especificação Técnica Aprofundada

Documento 2 de 2 — **Deep Dive**

Versão 1.0 · Abril/2026

Este documento detalha cada fase do roadmap: arquitetura, modelos de dados, fluxos, algoritmos, regras de parsing por org, eventos do gateway, política de rate-limit, telemetria e operação. Onde fizer sentido, há pseudo-código e exemplos de schema.

Sumário

1. Visão de Arquitetura
2. Painel Web (Frontend)
3. Backend / API & Autenticação
4. Banco de Dados
5. Worker — Núcleo do Selfbot
6. Descoberta de Orgs, Categorias e Canais
7. Parser de Embeds & Botões
8. Motor de Filas (Round-Robin)
9. Detecção de Partida
10. Mensagens dentro da Partida
11. Telemetria, Stats & Logs
12. Rate-Limit, Erros e Resiliência
13. Multi-Instância & Rotação de Tokens
14. Operação, Backups e Hardening

1. Visão de Arquitetura

O sistema é composto por três processos lógicos que podem rodar no mesmo host ou em hosts separados: **API/Web**, **Worker** e **Banco**. A comunicação em tempo real entre Worker → API → Painel é feita via WebSocket; configuração persiste em banco relacional; segredos (tokens) ficam criptografados em repouso.

Fluxo principal

- 1) Operador abre o painel, faz login e configura tokens, orgs, modos, mensagens e delay.
- 2) Operador clica em Start na instância (BOT1 ou BOT2).
- 3) API publica o evento de start e o Worker carrega a configuração.
- 4) Worker abre uma sessão de gateway por token e descobre guilds/canais.
- 5) O motor de filas calcula o próximo (org, modo) a entrar, respeitando limites.
- 6) Worker clica no botão correto da fila (Entrar / Gel Normal / Jogar Normal etc.).
- 7) Quando a partida é criada, Worker detecta o canal e envia a mensagem configurada.
- 8) Stats e logs são empurrados para o painel em tempo real.

Stack sugerida (alto nível)

Componente	Tecnologia sugerida	Motivo
Frontend	React + Vite + Tailwind	Reproduz o tema escuro do painel atual.
Backend API	Node.js (Fastify/Express) ou Python (FastAPI)	WebSocket nativo e ecossistema rico.
Worker	Node.js com cliente HTTP+WS próprio	Acesso fino ao gateway/REST do Discord.
Banco	PostgreSQL	Transacional, JSONB para configs flexíveis.
Cache/Fila	Redis (opcional)	Buckets de rate-limit e pub/sub entre instâncias.

2. Painel Web (Frontend)

Reproduz o painel “IMPERIUNS — PAINEL DE CONTROLE”. Tema escuro, cards arredondados, destaques em verde para status saudável, vermelho para Stop e azul para acentos.

Telas e componentes

- **Header de status:** badges “Conectado” e “Rodando”, abas de instância (BOT1/BOT2) e indicação “Instância ativa”.
- **Controle:** grande botão circular (Stop em vermelho quando rodando, Start em verde quando parado) com texto contextual.
- **Stats:** cards Entradas, Na Fila, Partidas, DMs, Uptime, com contadores em destaque e identificação do usuário Discord conectado (@handle).
- **Reset Stats:** botão para zerar contadores da instância sem afetar configuração.
- **Configuração:** textarea de tokens (até 5, um por linha), rotação em minutos, categoria (Mobile/Misto/Emulador), checkboxes de orgs (Surf, Tokio, Paris, Panda, REAL, CIPHER...), delay em segundos, modos permitidos (1x1/2x2/3x3/4x4 e variações), mensagem dentro da partida com variáveis e mensagem secundária por org.
- **Logs:** console rolante com no mínimo 100 linhas, níveis coloridos (INFO, WARN, ERROR), timestamp e botão Limpar.
- **Rodapé:** aviso de uso por conta e risco.

Variáveis suportadas nas mensagens

```
{adversary_mention} -> <@USER_ID>
{adversary_id}      -> 1234567890
{channel_name}      -> fila-958309 / partida-0
{org_name}          -> Colombia / Moreira / Gold ...
{mode}              -> 1x1 / 2x2 / 3x3 / 4x4
{format}            -> Mobile / Misto / Emulador
{value}             -> R$ 5,00
```

Mensagem secundária por org

Aceita formato em pares “org | mensagem”. A prioridade é por nome da org ou guild_id. Se houver match com a org da partida, sobrescreve a mensagem global.

```
Helipa | teste
Moreira F1 | oi {adversary_mention}
1395120020681392138 | mensagem dedicada por guild_id
```

3. Backend / API & Autenticação

API protegida por sessão (cookie httpOnly) com hashing de senha do operador. Toda requisição que altera configuração exige usuário autenticado.

Endpoints essenciais

Método	Rota	Descrição
POST	/api/auth/login	Login do operador.
POST	/api/auth/logout	Encerra sessão.
GET	/api/instances	Lista instâncias (BOT1, BOT2...) e status.
POST	/api/instances/:id/start	Inicia a instância.
POST	/api/instances/:id/stop	Para a instância.
POST	/api/instances/:id/reset-stats	Zera contadores.
GET	/api/config/:id	Retorna configuração da instância.
PUT	/api/config/:id	Salva configuração (tokens, orgs, modos, mensagens, delay, rotação).
GET	/api/orgs	Catálogo de orgs conhecidas.
GET	/api/logs/:id?since=ts	Histórico de logs paginado.
WS	/ws/:id	Stream em tempo real: stats, logs e eventos do worker.

Eventos via WebSocket

```
{"type": "stats", "payload": {"entradas": 1733, "naFila": 7, "partidas": 42, "dms": 12, "uptime": 34}}
{"type": "log", "payload": {"ts": "21:30:43", "level": "ERROR", "msg": "Erro ao buscar tópicos da guild 134303"}}
{"type": "status", "payload": {"instance": "BOT1", "connected": true, "running": true, "user": "@sophyymaria"}}
{"type": "queue", "payload": {"org": "Colombia", "mode": "2x2", "action": "join", "value": "R$5,00"}}
{"type": "match", "payload": {"org": "Colombia", "channel": "fila-958309", "adversary": "<@149...>"}}
```

4. Banco de Dados

Esquema mínimo (PostgreSQL):

```
users(id, email, password_hash, created_at)
instances(id, user_id, name, running, created_at)
tokens(id, instance_id, ciphertext, status, last_used_at, rotation_minutes)
orgs(id, guild_id, name, category, max_queues, priority, enabled)
channels(id, org_id, name, mode, format, kind) -- kind: 'queue' | 'match' | 'category'
queues_active(id, instance_id, token_id, org_id, channel_id, mode, joined_at)
matches(id, instance_id, token_id, org_id, channel_id, adversary_id, created_at, msg_sent)
messages(instance_id, scope, org_id NULL, body, image_url NULL) -- scope: 'global' | 'org'
stats_daily(instance_id, day, entradas, partidas, dms)
logs(id, instance_id, ts, level, source, message)
```

Notas:

- **tokens.ciphertext**: criptografia simétrica em repouso (chave fora do banco).
- **orgs.max_queues**: limite por org (configurado, calibrado por observação).
- **queues_active**: usado pelo motor para saber quantas filas o bot mantém ativas.
- **matches.msg_sent**: garante idempotência de envio.

5. Worker — Núcleo do Selfbot

O Worker é responsável por manter conexões WebSocket com o gateway do Discord para cada token ativo, processar eventos relevantes e expor uma API interna para o motor de filas e o detector de partidas.

Eventos do gateway que importam

Evento	Uso
READY	Identifica usuário, popula cache de guilds/canais e marca token como “Conectado”.
GUILD_CREATE / GUILD_DELETE	Atualiza orgs disponíveis em runtime.
CHANNEL_CREATE / CHANNEL_DELETE	Atualiza canais e remove partidas.
MESSAGE_CREATE / MESSAGE_UPDATE	Atualiza estado das filas (cards de Fila de Competição) e atualiza estado.
GUILD_MEMBER_ADD	Pode complementar a detecção do adversário em alguns formatos.
RESUMED / RECONNECT	Restaura estado sem perder filas ativas.

Sessão por token (estados)

```
DISCONNECTED -> CONNECTING -> IDENTIFYING -> READY
READY -> RUNNING (recebendo eventos)
RUNNING -> RATE_LIMITED (backoff) -> RUNNING
RUNNING -> RESUMING -> RUNNING (em caso de queda)
qualquer -> INVALID (401/403, token quebrado) -> isolado e reportado
```

6. Descoberta de Orgs, Categorias e Canais

Para cada guild que o token participa, o Worker mapeia categorias (MOBILE, MISTO, EMULADOR) e canais cujo nome corresponde aos modos configurados.

Heurística de nome de canal

Padrões aceitos (case-insensitive, com ou sem emojis/decorações):

```
^(\d)x(\d)-(mob|mobile|misto|emu|emulador)$
```

```
^(\d)v(\d)-(mob|mobile|misto|emu|emulador)$
```

Exemplos casados: 1x1-mob, 2x2-mobile, 3x3-misto, 4x4-emu

Categorias

A categoria configurada no painel filtra quais canais o bot considera. Exemplo: ao escolher “Mobile”, o bot só atua em canais sob a categoria MOBILE de cada guild selecionada.

Cache e invalidação

- Cache em memória por guild_id com TTL curto (ex.: 60s) e refresh sob demanda.
- Invalidação imediata em CHANNEL_CREATE/UPDATE/DELETE da guild.
- Backoff e retry no caso do erro real visto nos logs (HTTP 429 ao buscar canais/tópicos).

7. Parser de Embeds & Botões

Cada org publica um card de “Fila de Competição” com leves variações. O Worker precisa interpretar esses cards e identificar qual botão clicar. A solução é um conjunto de **adaptadores** com regras por org/template.

Modelo unificado de fila

```
Queue = {
  org_id, channel_id, message_id,
  mode: '1x1' | '2x2' | '3x3' | '4x4',
  format: 'Mobile' | 'Misto' | 'Emulador',
  value: number, // R$
  players: [user_id, ...], // pode estar vazio
  buttons: [
    { id, label, style, action }
    // ações possíveis: 'join', 'join_normal', 'join_full_ump_xm8',
    // 'join_gel_normal', 'join_gel_inf', 'leave'
  ]
}
```

Variações observadas

Org / Template	Botões característicos	Observações
Moreira (1x1 mobile)	Gel Normal, Gel Infinito, Sair da Fila	Distingue tipo de gel; usar action 'join_gel_normal' ou 'join_gel_inf'.
Colombia (1x1-mob)	Gel Normal, Gel Inf, Sair	Mesma lógica de gel.
Colombia (2x2-mob)	Entrar, Sair	Card mais simples, apenas join/leave.
Gold Apostas (3x3, 4x4)	Jogar Normal, Jogar Full UMP & XM8, Sair da Fila	Dois modos no mesmo card.
Bounty (2x2 full mobile)	Estilo similar a Moreira	Variante visual no embed; lógica equivalente.

Diretriz: o adaptador **NUNCA** hardcoda o custom_id do botão. Ele identifica pelo **label** (com normalização) e pelo estilo (verde/cinza/vermelho), porque custom_ids podem mudar entre updates do bot da org.

8. Motor de Filas (Round-Robin Org × Modo)

Regra central: o bot não prefere filas com player nem sem player. Ele reveza entre todas as orgs ativas e todos os modos permitidos para **maximizar a quantidade de filas simultâneas**, respeitando o limite individual de cada org.

Algoritmo (pseudo-código)

```
loop:
    candidates = []
    for org in enabled_orgs:
        if active_queues[org] >= org.max_queues: continue
        for mode in allowed_modes:
            queue = pick_next_queue(org, mode) # qualquer fila utilizável,
                                                # com ou sem player
            if queue is not None:
                candidates.append((org, mode, queue))

    # Round-robin justo: ordena por (last_action_ts ASC) por (org, mode)
    candidates.sort(key=lambda c: last_action_ts[(c.org, c.mode)])

    if candidates:
        org, mode, queue = candidates[0]
        click(queue.button_for(operator_choice)) # ex.: gel_normal
        active_queues[org] += 1
        last_action_ts[(org, mode)] = now()
        sleep(delay + jitter())
    else:
        sleep(short_idle)
```

Liberação de slot

- Quando a fila vira partida (canal de match criado), o slot é liberado.
- Quando a fila é cancelada/expirada, o slot também é liberado.
- Quando o bot clica em Sair (manual ou por regra), libera o slot.

Distribuição multi-token

Se houver múltiplos tokens ativos, cada token tem seu próprio contador por org. O round-robin acontece por token. Tokens não competem entre si pela mesma fila.

9. Detecção de Partida

Quando o bot da org confirma o match, um canal é criado (ou o token é adicionado a ele). O Worker captura via CHANNEL_CREATE e/ou recebendo a primeira mensagem nesse canal.

Padrões de nome

```
Categoria: 'Partidas' ou 'SUA PARTIDA N'
Canais:
  ^fila-\d+$      ex.: fila-958309, fila-958285
  ^partida-\d+$   ex.: partida-0
```

Identificação do adversário

- Lê membros do canal recém-criado e remove o próprio bot da lista.
- Em fallback, parseia menção dentro da primeira mensagem do bot da org.
- Mantém um buffer curto para evitar race condition (canal criado antes do membro ser adicionado).

Idempotência: antes de enviar a mensagem, o Worker verifica em *matches* se já existe registro com (instance_id, channel_id) e msg_sent=true.

10. Mensagens dentro da Partida

Pipeline de envio:

- 1. Resolver template: se houver mensagem secundária para a org da partida, usa ela; senão usa a global.
- 2. Substituir variáveis: {adversary_mention}, {adversary_id}, {channel_name}, {org_name}, {mode}, {format}, {value}.
- 3. Anexar imagem (se configurada).
- 4. Enviar com retry leve (1 tentativa extra) em caso de 5xx; em 429 respeita o Retry-After.
- 5. Marcar match.msg_sent = true para idempotência.

Exemplo de payload final

```
POST /channels/<channel_id>/messages
{
  "content": "Oii Mocoo, me manda uma mensagem no <@1499110161531277362>",
  "allowed_mentions": {"parse": ["users"]}
}
(+ multipart com imagem se aplicável)
```

11. Telemetria, Stats & Logs

Métricas que aparecem no painel:

Métrica	Como é incrementada
Entradas (filas entradas)	+1 a cada clique bem-sucedido em botão de entrar.
Na Fila (filas ativas)	Tamanho atual de <code>queues_active</code> da instância.
Partidas encontradas	+1 quando um match é detectado e associado.
DMs detectadas	+1 quando o adversário envia DM ao bot (opcional, derivado de <code>MESSAGE_CREATE</code> em DM).
Uptime	Tempo desde o último Start da instância.
@usuário	Username do token ativo no momento.

Logs

Estrutura: **[hh:mm:ss] [LEVEL] origem · mensagem**. Origem pode ser: `token:{username}`, `org:{nome}`, `channel:{nome}`, `action:{join|leave|match|send}`. Manter no mínimo 100 linhas no painel e persistir em banco com rotação por dia.

12. Rate-Limit, Erros e Resiliência

O Discord retorna cabeçalhos X-RateLimit-* em quase toda chamada REST. O Worker mantém buckets por rota e por token. Em 429, respeita Retry-After e enfileira a ação.

Tipos de erro e tratamento

Erro	Significado	Ação
401	Token inválido	Marcar token como INVALID, parar suas filas, alertar painel.
403	Sem permissão / bloqueado	Desativar org no token; logar.
429	Rate-limit	Aguardar Retry-After + jitter. Bucket isolado por token+rota.
5xx	Falha temporária do Discord	Retry exponencial limitado.
WS close 4007	Token inválido no gateway	Marcar INVALID.
WS close 4014	Intents não permitidos	Não aplicável a selfbot, mas tratar genericamente.

Os logs reais já mostram **HTTP 429** ao buscar canais/tópicos. O parser deve ler Retry-After e **nunca** bloquear a thread principal — todas as ações REST passam por uma fila com leaky-bucket por token.

13. Multi-Instância & Rotação de Tokens

O painel mostra duas instâncias (BOT1, BOT2). Cada instância tem sua própria configuração, conjunto de tokens, contadores e logs. Instâncias rodam em isolamento.

Rotação automática

- Configuração informa intervalo em minutos (ex.: 90).
- Quando expira, o Worker desconecta limpo o token corrente, libera filas e abre o próximo do ciclo.
- Se a textarea de tokens estiver vazia, mantém os tokens já salvos no servidor.
- Painel mostra “Tokens ativos: X/Y · Próxima troca em Mm Ss”.

Comportamento ao desconectar

Antes de trocar de token, o Worker tenta sair das filas ativas para não deixar “fantasmas”. Se falhar (rate-limit ou indisponibilidade), registra dívida e tenta novamente quando o token reentrar no ciclo.

14. Operação, Backups e Hardening

Itens não-funcionais essenciais antes de uso pesado:

- **Backup de configuração:** export/import JSON pelo painel.
- **Health-check:** endpoint /health para monitoramento externo.
- **Auto-restart:** supervisor reinicia o Worker em crashes inesperados.
- **Rotação de logs:** arquivar diariamente, manter últimos N dias.
- **Segurança do painel:** rate-limit no login, senha forte do operador, sessão expira.
- **Criptografia de tokens:** chave fora do banco; nunca logar token completo.
- **Aviso visível:** rodapé já indica risco de uso (selfbot viola ToS do Discord).

Definição de Pronto (Definition of Done)

- Bot conecta tokens, descobre orgs/canais e entra em filas conforme configuração.
- Atinge limite de filas por org via round-robin entre orgs e modos.
- Detecta partidas em todas as variações de nome e categoria.
- Envia exatamente uma mensagem por partida, com variáveis e mensagem por org.
- Painel reflete em tempo real stats, logs e status; Start/Stop funcionam.
- Tokens inválidos, 429 e quedas de gateway são contidos sem afetar o restante.